UNIVERSITÀ DELLA CALABRIA

Anno accademico 2026/2027 – Appello di gennaio

Progetto d'esame

Algorithmic Game Theory

Documento di lavoro: requisiti, metodo, idee, decisioni

Aggiornato fino alla Lezione 6 · Ultima compilazione: 5 novembre 2026

Indice

1 Cosa sappiamo dell'esame **1**

2 Come impostare un progetto AGT **2**

2.1 Le sette domande della formalizzazione 2

2.2 Dall'implementazione alla relazione 2

2.3 Checklist prima della consegna 3

3 Banca delle idee **4**

3.1 Giochi combinatori (Lezione 1) 4

3.2 Teoria dell'utilità e decisioni sotto incertezza (Lezione 2) 5

3.3 Giochi non cooperativi (Lezioni 3 e 4) 5

3.4 Giochi coalizionali (Lezione 5) 7

3.5 Giochi coalizionali: aspetti computazionali (Lezione 6) 8

3.6 Argomenti ancora da affrontare 9

4 Domande aperte per il docente **10**

5 Piano di lavoro e decisioni **11**

5.1 Calendario indicativo 11

5.2 Strumenti di studio in programma 11

5.3 Registro delle decisioni 12

CAPITOLO 1

Cosa sappiamo dell'esame

L'unica informazione ufficiale finora (slide della Lezione 1):

> **Definizione 1.1 (Esame).** L'esame consiste in un **progetto** in cui si deve **formalizzare** e **implementare** la soluzione di un problema usando gli strumenti dell'Algorithmic Game Theory.

Il progetto è **individuale** (indicazione del docente, novembre 2026). Il materiale del corso non indica quale problema affrontare: la slide parla genericamente di "un problema".

Due verbi, due metà del lavoro:

- **formalizzare**: trasformare un problema a parole in un modello preciso (giocatori, azioni, utilità, concetto di soluzione);
- **implementare**: scrivere codice che calcola quella soluzione, e mostrare che funziona.

> **Attenzione (cosa non sappiamo ancora).** Formato della consegna, chi sceglie il problema, scadenze e criteri di valutazione non sono ancora stati comunicati. Le domande da fare al docente sono raccolte nel Capitolo 4. Finché non ci sono risposte, tutto ciò che segue è una *proposta* di metodo, non un requisito.

CAPITOLO 2

Come impostare un progetto AGT

Qualunque sia il tema, un progetto di AGT segue la stessa sequenza. Conviene usarla anche come scaletta della relazione finale.

2.1 Le sette domande della formalizzazione

1. **Problema**: qual è la situazione reale? Perché è interessante?
2. **Giocatori**: chi prende decisioni? Quanti sono? ($N = \{1, \ldots, n\}$)
3. **Azioni / strategie**: cosa può fare ciascuno? (A_i, oppure mosse in uno stato)
4. **Informazione e tempo**: si muove a turni o simultaneamente? Ognuno sa tutto (informazione perfetta) o no? C'è casualità?
5. **Utilità**: cosa vuole ciascun giocatore? (u_i, oppure $v(S)$ per le coalizioni). È un gioco a somma zero?
6. **Concetto di soluzione**: cosa vogliamo calcolare? (valore minimax, equilibrio di Nash, valore di Shapley, matching stabile, vincitore di un'elezione, . . .)
7. **Algoritmo**: come lo calcoliamo? Con che complessità? Si può fare in modo esatto o serve un'approssimazione o un'euristica?

Esempio (il tris del Lab 1 formalizzato).

Problema	giocare a tris in modo ottimale.
Giocatori	2: MAX (X) e MIN (O).
Azioni	piazzare il proprio segno in una cella vuota.
Informazione	a turni alternati, informazione perfetta, niente caso.
Utilità	$+1$ / 0 / -1 per MAX; somma zero.
Soluzione	valore minimax di ogni stato, e la mossa che lo realizza.
Algoritmo	Minimax con potatura alpha-beta; l'albero completo ha 549 946 nodi.

2.2 Dall'implementazione alla relazione

Dopo la formalizzazione:

8. **Implementazione**: codice pulito, eseguibile, con esempi e test.
9. **Esperimenti**: misure che mostrano che l'algoritmo funziona e quanto costa (tempo, nodi visitati, qualità della soluzione) al crescere dell'istanza.
10. **Discussione**: limiti del modello (quali ipotesi sono irrealistiche?) e del metodo (dove non scala?), possibili estensioni.

Osservazione (cosa rende un progetto convincente).

- Un confronto: algoritmo base contro versione migliorata, oppure due concetti di soluzione sullo stesso problema. Esempio già visto: Minimax contro alpha-beta, 549 946 nodi contro 18 297.
- Numeri misurati, non solo affermati.
- Ipotesi del modello dette esplicitamente, con il motivo per cui sono ragionevoli.

2.3 Checklist prima della consegna

- ☐ Ogni elemento del modello (giocatori, azioni, utilità) è definito in modo preciso.
- ☐ Il concetto di soluzione è motivato: perché proprio questo?
- ☐ L'algoritmo è descritto (pseudocodice) e se ne discute la complessità.
- ☐ Il codice gira da zero seguendo le istruzioni.
- ☐ Ci sono esperimenti con tabelle o grafici e un commento ai risultati.
- ☐ Ci sono limiti ed estensioni.

CAPITOLO 3

Banca delle idee

Le idee sono ordinate per argomento del corso. Ogni lezione "sblocca" nuovi strumenti: un'idea diventa praticabile quando il suo argomento è stato spiegato.

3.1 Giochi combinatori (Lezione 1)

Strumenti disponibili: albero di gioco, Minimax, alpha-beta, e le tecniche per giochi grandi (profondità limitata, euristiche, *transposition table*, *move ordering*).

Idea 1: IA per Forza 4

DA LEZIONE 1

- **Modello**: identico al tris (2 giocatori, turni, informazione perfetta, somma zero), ma su una griglia 6×7 con gravità.
- **Perché è interessante**: l'albero completo è troppo grande per Minimax puro, quindi servono tutte le tecniche della slide "Oltre alpha-beta".
- **Esperimenti**: nodi visitati e tempo al variare della profondità; effetto di *move ordering* (colonne centrali prima) e della *transposition table*; percentuale di vittorie contro un giocatore casuale e uno "greedy".
- **Rischio**: basso. È un'estensione diretta del Lab 1.

Idea 2: tris generalizzato (m, n, k)

DA LEZIONE 1

- **Modello**: griglia $m \times n$, vince chi allinea k segni (il tris è $(3, 3, 3)$, il *gomoku* è $(15, 15, 5)$).
- **Perché è interessante**: permette di misurare come esplode l'albero di gioco al crescere di m, n, k, e da che punto la ricerca esatta smette di essere praticabile.
- **Esperimenti**: per istanze piccole, valore esatto del gioco (vince il primo o si pareggia?); per istanze grandi, ricerca a profondità limitata con euristica.
- **Rischio**: basso-medio. La parte esatta è facile; l'euristica richiede tentativi.

Idea 3: IA per Othello (Reversi) o Dama

DA LEZIONE 1

- **Modello**: gioco combinatorio con regole di mossa più ricche.
- **Perché è interessante**: la funzione di valutazione euristica diventa centrale (mobilità, angoli, stabilità dei pezzi in Othello).
- **Rischio**: medio. Implementare correttamente le regole richiede tempo.

> **Osservazione (limite dei giochi combinatori come progetto).** Queste idee usano solo la prima parte del corso. Il resto del programma (Nash, Shapley, voto, matching, meccanismi, divisione equa) offrirà probabilmente temi più vicini allo spirito "formalizzare un problema reale". Conviene tenerle come piano B e decidere più avanti.

3.2 Teoria dell'utilità e decisioni sotto incertezza (Lezione 2)

Strumenti disponibili: relazioni di preferenza, utilità ordinale e cardinale (VNM), lotterie e utilità attesa, atteggiamento verso il rischio, ricerca bayesiana (massimizzazione dell'utilità attesa con posteriore aggiornato).

Idea 4: ricerca bayesiana per un problema di *search and rescue*

DA LEZIONE 2

- **Modello**: un solo decisore contro l'incertezza. Un oggetto (relitto, persona dispersa, guasto in una rete) si trova in una cella sconosciuta con un prior non uniforme; ogni ispezione ha un costo e una probabilità di *falso negativo*.
- **Perché è interessante**: generalizza direttamente il Lab 2 e lo avvicina al caso reale dei paper in `labs/02-battleship/papers/` (volo AF 447). Permette di introdurre cose che nel lab non ci sono: rilevazione imperfetta, costi di spostamento diversi per cella, vincolo di budget.
- **Concetto di soluzione**: politica che massimizza l'utilità attesa; confronto tra la politica *greedy* (massima probabilità) e una che tiene conto del costo.
- **Esperimenti**: turni o costo medio per trovare l'oggetto, al variare del prior, del tasso di falsi negativi e del budget; confronto con le euristiche del Lab 2.
- **Rischio**: basso. Il codice del Lab 2 è già una base funzionante.

Idea 5: atteggiamento verso il rischio in un problema di allocazione

DA LEZIONE 2

- **Modello**: più agenti con funzioni di Bernoulli diverse ($u(x) = x$, $u(x) = \sqrt{x}$, $u(x) = x^2$) devono scegliere tra portafogli o contratti rischiosi.
- **Perché è interessante**: mostra in modo misurabile che la soluzione ottima dipende dalla *curvatura* di u e non dal valore atteso, e permette di calcolare equivalenti certi e premi per il rischio.
- **Collegamento**: è il ponte naturale verso i giochi coalizionali (il fondo comune della lezione è già una coalizione) e verso il *mechanism design* (le assicurazioni).
- **Rischio**: medio. Da solo è un po' esile: funziona meglio come *componente* di un progetto più grande, una volta visti Nash o Shapley.

3.3 Giochi non cooperativi (Lezioni 3 e 4)

Strumenti disponibili: forma normale, dominanza e IESDS, best response, equilibri di Nash puri e misti, condizione del supporto, minimax con programmazione lineare, giochi potenziali e dinamica di best response, equilibri correlati con programmazione lineare, fictitious play e regret matching. Codice già scritto e verificato nei laboratori 3 e 4.

Idea 6: giochi di sicurezza (pattugliamento) DA LEZIONE 3

- **Modello**: un difensore con k risorse (pattuglie, guardie) deve proteggere n bersagli; un attaccante sceglie quale colpire. Ogni bersaglio ha un valore diverso per i due giocatori. È uno degli esempi di applicazione citati nella lezione 1.
- **Perché è interessante**: è un problema reale (aeroporti, porti, parchi naturali) e mostra il valore della randomizzazione: un difensore prevedibile viene aggirato. Nel caso a somma zero il minimax del laboratorio 3 risolve direttamente il problema.
- **Concetto di soluzione**: strategia minimax del difensore; confronto con la strategia "ingenua" che protegge sempre i bersagli più preziosi.
- **Esperimenti**: perdita attesa al variare di k e di n; tempo del programma lineare quando le strategie pure del difensore (sottoinsiemi di k bersagli) esplodono combinatoriamente, il che porta naturalmente a rappresentazioni compatte.
- **Rischio**: basso-medio. La base (minimax con `linprog`) c'è già; la parte non banale è la crescita del numero di strategie.

Idea 7: rilevamento di comunità come gioco potenziale DA LEZIONE 3

- **Modello**: il gioco del laboratorio 3, con nodi che scelgono un cluster. Si estende con utilità diverse (per esempio penalizzando i cluster troppo grandi, per evitare l'equilibrio banale "tutti insieme").
- **Perché è interessante**: il laboratorio ha mostrato che l'equilibrio raggiunto dipende dal punto di partenza e che il massimo del potenziale è la soluzione sbagliata. Studiare quando la dinamica trova le comunità giuste è una domanda vera.
- **Esperimenti**: modelli a blocchi stocastici con rapporto p/q variabile; accuratezza (per esempio con l'indice NMI) e numero di turni; confronto con Louvain di `networkx`.
- **Rischio**: basso. Il codice c'è e gira in pochi secondi su 400 nodi.

Idea 8: apprendimento negli equilibri DA LEZIONE 4

- **Modello**: giochi ripetuti; giocatori che usano fictitious play, regret matching o varianti, anche uno contro l'altro.
- **Perché è interessante**: il laboratorio 4 ha mostrato che una variante del regret matching (quella delle slide) converge al punto sbagliato su un gioco asimmetrico. Un confronto sistematico (giochi a somma zero, coordinamento, giochi con equilibri correlati diversi da Nash) dice quando e a che cosa converge ogni algoritmo.
- **Esperimenti**: distanza dall'equilibrio nel tempo; distribuzione congiunta del regret matching confrontata con il politopo degli equilibri correlati calcolato con programmazione lineare; giochi con cicli (Shapley 3×3, dove fictitious play non converge).
- **Rischio**: basso. Molto "sperimentale", meno "modellazione di un problema reale": da chiarire se è nello spirito richiesto.

> **Osservazione (il semaforo come progetto).** Il calcolo degli equilibri correlati con l'obiettivo maximin ha restituito esattamente il semaforo nell'attraversamento. Un progetto che progetta dispositivi di coordinamento (incroci, accesso a un canale radio, turni di utilizzo di una risorsa) mettendo a confronto benessere ed equità sarebbe una buona sintesi delle lezioni 3 e 4.

3.4 Giochi coalizionali (Lezione 5)

Strumenti disponibili: giochi in funzione caratteristica, superadditività, struttura di coalizione con programmazione dinamica, core, nucleolo (Kopelowitz), valore di Shapley. Codice già scritto e verificato nel laboratorio 5, compresi la programmazione dinamica e il nucleolo.

Idea 9: ripartizione equa di costi o ricavi

DA LEZIONE 5

- **Modello**: un problema concreto di condivisione, per esempio aziende che condividono un impianto, utenti che si dividono la bolletta di un servizio comune, comuni che finanziano un'infrastruttura. La funzione caratteristica si ricava da un modello di costo (spesso analitico).
- **Perché è interessante**: permette di confrontare sullo stesso problema tre concetti di soluzione (core, nucleolo, Shapley) e di misurare il conflitto tra stabilità ed equità, visto più volte nella lezione.
- **Esperimenti**: il core è vuoto? Shapley vi appartiene? Quanto distano nucleolo e Shapley? Tempi di calcolo al crescere di n e approssimazione di Shapley per campionamento di permutazioni quando 2^n diventa troppo.
- **Rischio**: basso-medio. La parte delicata è costruire una funzione caratteristica credibile.

Idea 10: valore di Shapley per spiegare un modello (SHAP)

DA LEZIONE 5

- **Modello**: le feature di un modello di machine learning sono i giocatori, $v(C)$ è la qualità della predizione usando solo le feature in C. È l'applicazione citata nella lezione 1.
- **Perché è interessante**: collega il corso al machine learning e pone subito il problema computazionale (con 20 feature, 2^{20} coalizioni da valutare).
- **Esperimenti**: Shapley esatto su un dataset piccolo, confronto con l'approssimazione per campionamento e con la libreria `shap`; verifica degli assiomi (efficienza, giocatore nullo) sui risultati.
- **Rischio**: medio. Va definito con cura che cosa vuol dire "usare solo le feature in C".

Idea 11: struttura di coalizione per formare squadre

DA LEZIONE 5

- **Modello**: assegnare persone a gruppi di lavoro (o veicoli a rotte, o server a compiti) quando il valore di un gruppo non è additivo. Non superadditivo: gruppi troppo grandi rendono meno.
- **Perché è interessante**: la programmazione dinamica esatta in $O(3^n)$ diventa impraticabile già con poche decine di agenti; si confronta con euristiche (greedy, clustering, simulated annealing), misurando quanto ci si allontana dall'ottimo dove questo si può ancora calcolare.
- **Rischio**: medio. Aiuta avere già il codice esatto per avere un riferimento.

3.5 Giochi coalizionali: aspetti computazionali (Lezione 6)

Strumenti disponibili: gioco dell'aeroporto (Shapley in $O(n)$), giochi su sottografo indotto, MC-Nets (anche con letterali negativi), approssimazione Monte Carlo del valore di Shapley. Codice scritto e verificato contro la formula esatta nel laboratorio 6.

> **Osservazione (le idee 9–11 diventano più solide).** La lezione toglie il limite principale delle idee 9 e 10: con Monte Carlo il valore di Shapley si stima per qualunque gioco con decine o centinaia di giocatori, e gli errori misurati nel laboratorio (circa 3% con $m = 10\,000$ e 20 giocatori) danno già un'idea di quanti campioni servono. Per l'idea 10 (SHAP) è esattamente il metodo usato nella pratica.

Idea 12: ripartire il costo di un'infrastruttura condivisa

DA LEZIONE 6

- **Modello**: utenti con esigenze annidate che condividono una struttura (spazio in cloud con quote diverse, banda, una condotta d'acqua, livelli di abbonamento a un software). Il caso base è il gioco dell'aeroporto; le estensioni rompono la sua formula chiusa: più utenti per livello, costi non lineari, una rete ad albero invece di una sola pista (ogni utente usa un percorso dalla radice).
- **Perché è interessante**: è un problema reale di ripartizione equa con una soluzione esatta nota per il caso base, che fa da verifica. Nelle estensioni la formula non vale più e servono Monte Carlo o una MC-Net con regole disgiuntive.
- **Esperimenti**: Shapley contro ripartizione proporzionale all'uso e contro parti uguali; verifica che la ripartizione stia nel core (nessun gruppo pagherebbe meno da solo); nucleolo per confronto; tempi e precisione al crescere degli utenti.
- **Rischio**: basso. Il codice del lab copre il caso base; il notebook cita due paper sulle estensioni (Vázquez-Brage et al. 1997, Hou et al. 2018).

Idea 13: stimare Shapley in fretta: confronto fra campionatori

DA LEZIONE 6

- **Modello**: nessun problema reale; si confrontano metodi di stima del valore di Shapley (Monte Carlo semplice, *antithetic sampling*, campionamento stratificato per posizione) su giochi di cui si conosce il valore esatto (aeroporto, ISG, MC-Nets).
- **Perché è interessante**: la lezione cita le tecniche di riduzione della varianza senza mostrarle (riferimento: Mitchell et al., 2022). Il lab ha mostrato che l'errore relativo pesa sui giocatori con quota piccola, un punto da cui partire.
- **Esperimenti**: errore contro numero di campioni e contro tempo, per giocatore e in media; quando conviene ciascun metodo.
- **Rischio**: basso, ma è molto "sperimentale" (come l'idea 8): da chiarire se è nello spirito di "formalizzare un problema". Meglio come componente dell'idea 10 o 12.

Idea 14: centralità di una rete come valore di Shapley

DA LEZIONE 6

- **Modello**: i nodi di una rete (sociale, di trasporto, di comunicazione) sono giocatori; il valore di un gruppo è quanto "copre" la rete (per esempio quanti nodi sono raggiunti dal gruppo o dai suoi

vicini). Il valore di Shapley diventa una misura di importanza che tiene conto delle sovrapposizioni fra nodi.

- **Perché è interessante**: in un ISG Shapley è solo metà del grado pesato (più il cappio), cioè niente di nuovo. Funzioni di copertura come quella sopra non sono ISG, e lì il valore di Shapley dà classifiche diverse dalle centralità classiche. Si collega all'idea 7 (comunità come gioco potenziale).
- **Esperimenti**: Shapley (con Monte Carlo) contro grado, betweenness e PageRank su reti reali di `networkx`; uso della classifica per scegliere chi vaccinare o dove mettere sensori, misurando la copertura ottenuta.
- **Rischio**: medio. Va scelta con cura una funzione di valore sensata.

3.6 Argomenti ancora da affrontare

Argomento	**Anticipazioni (possibili spunti)**
Scelta sociale computazionale	regole di voto, paradosso di Condorcet; *ensemble* di modelli
Matching markets	matching stabile; studenti–alloggi, donatori di rene, lavoratori–compiti
Mechanism design	aste, incentivi a dire il vero
Fair division	divisione equa di risorse

CAPITOLO 4

Domande aperte per il docente

Da chiarire il prima possibile, perché cambiano il modo di lavorare.

1. Il progetto è **individuale** o **di gruppo**? **RISOLTA**
   Individuale.
2. Il **problema** lo sceglie lo studente o viene assegnato? Serve un'approvazione preventiva della proposta? **APERTA**
   Nelle slide e nei lab non c'è un problema assegnato né un elenco di temi: sembra libero, ma va confermato.
3. Cosa si **consegna**: codice, relazione scritta, presentazione? Ci sono limiti di lunghezza? **APERTA**
4. C'è un **linguaggio** richiesto (Python, come nei lab)? **APERTA**
5. Quando va consegnato rispetto alla data dell'appello? **APERTA**
6. All'esame c'è anche una **discussione orale** o domande di teoria? **APERTA**
7. Quali sono i **criteri di valutazione**? **APERTA**

CAPITOLO 5

Piano di lavoro e decisioni

5.1 Calendario indicativo

Proposta da aggiornare quando saranno note le date ufficiali.

Periodo	Obiettivo
Ottobre	seguire le lezioni, aggiungere idee alla banca, avere risposta alle domande aperte
Novembre	**siamo qui**. Lavoro leggero: fare le domande aperte al docente e scegliere un tema provvisorio; continuare con gli appunti delle lezioni
Dicembre	seguire le ultime lezioni; prima di Natale avere tema confermato e formalizzazione abbozzata (Capitolo 2, domande 1–7)
Natale	**preparazione vera e propria**: implementazione, esperimenti, relazione; ripasso della teoria (con i quiz, se pronti)
Gennaio	rifinitura e preparazione della discussione, fino all'appello

5.2 Strumenti di studio in programma

Generatore di quiz per lezione FATTO

Pronto dal 2 novembre in `quiz/` (doppio clic su `Avvia quiz.bat`); si aggiunge una banca di domande a ogni lezione nuova. Il progetto iniziale era questo. Un applicativo snello per esercitarsi su ogni lezione. Non fa parte del progetto d'esame: serve a preparare la parte di teoria e a verificare di aver capito i laboratori.

- **Domande a risposta chiusa** (scelta multipla, vero/falso) scritte a partire dagli appunti: definizioni, enunciati dei teoremi e, soprattutto, i box *Attenzione*, che sono le trappole più probabili.
- **Esercizi generati**: numeri casuali e soluzione calcolata con il codice già verificato dei laboratori. Esempi: equilibrio misto di un gioco 2×2 casuale, best response a una strategia data, valore minimax, valore di Shapley e appartenenza al core di un gioco a 3 giocatori, lotteria equa e premio di assicurazione con $u(x) = \sqrt{x}$. Così gli esercizi non finiscono mai e la soluzione è sempre giusta.
- **Selezione**: per lezione, per tipo di domanda, o un mix di tutto il corso per il ripasso finale; a fine quiz il punteggio e, per ogni errore, il rimando alla sezione degli appunti da rileggere.
- **Forma minima**: le domande in un file di dati per lezione (facile da estendere quando arriva una lezione nuova) e un'interfaccia semplice, da riga di comando in Python oppure una pagina web locale. Da decidere quando si comincia.

- **Quando**: dopo aver fissato il tema del progetto, che ha la precedenza; le lezioni 1–5 si possono coprire subito, le altre man mano.

5.3 Registro delle decisioni

Data	Decisione
29/09/2026	Creato il documento di progetto. Nessun tema scelto.
03/10/2026	Riorganizzato il repository in `lessons/`, `labs/`, `documentazione/`. Aggiunta la Lezione 2 e due nuove idee (ricerca bayesiana, atteggiamento verso il rischio). Nessun tema scelto.
02/11/2026	Aggiunte le Lezioni 3–5 (giochi non cooperativi, equilibri correlati e apprendimento, giochi coalizionali) con i laboratori 3–5, e sei nuove idee (6–11). Aggiornata la Lezione 2 con le slide del Lab 2: con le navi affondate la ricerca bayesiana scende da 54 a 41–44 colpi. Nessun tema scelto.
02/11/2026	Messo in programma un generatore di quiz per lezione (strumento di studio, separato dal progetto d'esame). Verificato che nelle lezioni 2–5 e nei lab non si parla del progetto: l'unica informazione resta quella della Lezione 1.
02/11/2026	Deciso di concentrare la preparazione dell'esame nelle vacanze di Natale. Prima di allora: domande al docente e tema scelto, per arrivare a Natale pronti a implementare.
05/11/2026	Il progetto è individuale. Ricontrollato tutto il materiale delle lezioni 1–5: nessun problema assegnato né elenco di temi.
05/11/2026	Aggiunta la Lezione 6 (aspetti computazionali dei giochi coalizionali: aeroporto, ISG, MC-Nets, Monte Carlo) con il laboratorio 6 risolto e verificato; tre nuove idee (12–14). Neanche la lezione 6 parla del progetto. Nessun tema scelto.